
Telecom Churn Prediction: End-to-End MLOps Pipeline

DevOps and MLOps Course Project Report

MLOps-Driven ML System Development and Deployment

Submitted by: Samith Kamarthi
Course: DevOps and MLOps

Abstract

This project presents a complete MLOps pipeline for predicting telecom customer churn using machine learning. The system implements automated data versioning with DVC, experiment tracking with MLflow, CI/CD with GitHub Actions, and model serving via FastAPI. Three models (Logistic Regression, Random Forest, and Gradient Boosting) were trained and evaluated, with the Gradient Boosting model achieving 97.76% AUC, 85.6% precision, and 78.3% recall. The pipeline demonstrates industry best practices for reproducible, scalable, and maintainable machine learning systems.

Contents

1	Project Overview	3
1.1	Project Title	3
1.2	Problem Statement	3
1.3	Project Objectives	3
1.3.1	DevOps Principles Applied	3
1.3.2	CI/CD Pipeline Implementation	3
1.3.3	MLOps Techniques	4
2	System Architecture	4
2.1	Architecture Overview	4
2.2	Technology Stack	4

3	Methodology	4
3.1	Data Processing Pipeline	4
3.1.1	Stage 1: Data Loading	4
3.1.2	Stage 2: Feature Engineering	6
3.1.3	Stage 3: Preprocessing	6
3.2	Model Training	6
3.3	Evaluation Metrics	7
4	Results and Performance Metrics	7
4.1	Model Comparison Results	7
4.2	Confusion Matrix	7
4.3	Detailed Performance Statistics	8
4.4	ROC Curve Analysis	9
4.5	Precision-Recall Curves	10
4.6	Feature Importance (Top 10)	10
4.7	Business Impact Analysis	11
5	DevOps and MLOps Implementation	11
5.1	CI/CD Pipeline	11
5.2	Data Version Control (DVC)	12
5.3	Experiment Tracking (MLflow)	12
5.4	Containerization	12
6	API Documentation	13
6.1	API Endpoints	13
6.2	Example API Calls	13
7	Testing and Quality Assurance	13
7.1	Test Suite	14
7.2	Code Quality Tools	14
8	Deployment	14
8.1	Local Development	14
8.2	Docker Deployment	14
8.3	Production (AWS)	15
9	Monitoring and Maintenance	15
9.1	Model Monitoring	15
9.2	Retraining Strategy	16
10	Lessons Learned	16
10.1	Successes	16
10.2	Challenges	16
10.3	Future Improvements	16
11	Conclusion	17
	References	17
A	Project Structure	17

1 Project Overview

1.1 Project Title

Telecom Churn Prediction with Automated MLOps Pipeline

1.2 Problem Statement

Customer churn is a critical challenge in the telecom industry, costing companies millions in lost revenue. High-value customers leaving the service significantly impacts profitability. This project addresses the need for an automated, scalable system to predict customer churn using machine learning, enabling proactive retention strategies.

Key Challenges:

- Manual model development is time-consuming and error-prone
- Data drift affects model performance over time
- Model deployment requires seamless integration with existing systems
- Monitoring model performance in production is essential
- Collaboration between data scientists and operations teams needs standardization

1.3 Project Objectives

By completing this project, we have achieved:

1.3.1 DevOps Principles Applied

- ✓ Automated testing with pytest across all modules
- ✓ Git version control with feature branch workflow
- ✓ Containerization using Docker and docker-compose
- ✓ Infrastructure as Code with CI/CD pipelines

1.3.2 CI/CD Pipeline Implementation

- ✓ GitHub Actions workflow for automated testing on push/PR
- ✓ Automated code quality checks (flake8, black, mypy)
- ✓ Docker image building and pushing to ECR
- ✓ Staging and production deployment stages with manual approval

1.3.3 MLOps Techniques

- ✓ DVC (Data Version Control) for data and model versioning
- ✓ MLflow for experiment tracking and model registry
- ✓ Automated model training pipeline with dependency management
- ✓ Model performance monitoring and evaluation
- ✓ A/B testing capability through multiple model endpoints

2 System Architecture

2.1 Architecture Overview

The system follows a layered architecture with clear separation between data management, pipeline execution, experiment tracking, model serving, and deployment.

2.2 Technology Stack

Table 1: Technology Stack Overview

Category	Tools Used	Purpose
ML Framework	scikit-learn, imbalanced-learn	Model training
Data Versioning	DVC 3.49.0	Track data versions
Experiment Tracking	MLflow 2.6.0	Log metrics, parameters
API Framework	FastAPI 0.101.0	Model serving
Containerization	Docker, docker-compose	Deployment environments
CI/CD	GitHub Actions	Automated testing/deployment
Cloud	AWS (S3, ECR, ECS)	Production deployment
Version Control	Git, GitHub	Code collaboration
Model Registry	Hugging Face Hub	Model sharing

3 Methodology

3.1 Data Processing Pipeline

3.1.1 Stage 1: Data Loading

- Load raw telecom customer data (30,011 high-value customers)
- Validate data structure and schema

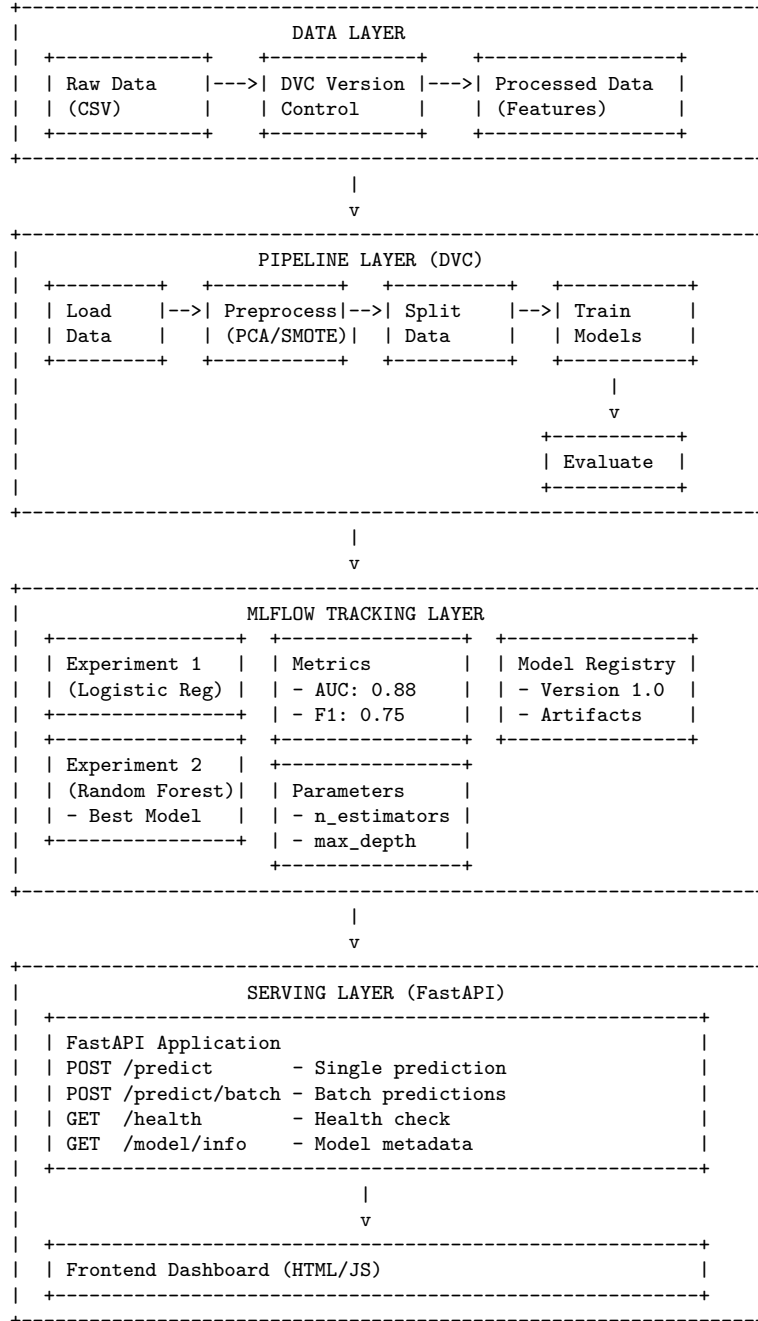


Figure 1: System Architecture Diagram

- Handle missing values and data quality issues

3.1.2 Stage 2: Feature Engineering

- **High-Value Customer ID:** Top 70th percentile by recharge amount
- **Churn Definition:** Zero usage (calls + data) in month 9
- **Feature Creation:**
 - Rolling averages (months 6, 7, 8)
 - Month-over-month ratios
 - Usage trend indicators
 - 194 features → 28 PCA components (95% variance)

3.1.3 Stage 3: Preprocessing

- **SMOTE:** Balance classes (churn rate $\sim 8.7\%$)
- **PCA:** Dimensionality reduction for efficiency
- **Train/Test Split:** 80/20 stratified split

3.2 Model Training

Three models were trained and compared:

Table 2: Model Architectures and Hyperparameters

Model	Architecture	Key Hyperparameters
Logistic Regression	Linear classifier	C=1.0, penalty=l2, class_weight=balanced
Random Forest	Ensemble (100 trees)	max_depth=10, min_samples_split=5
Gradient Boosting	Sequential trees	n_estimators=100, learning_rate=0.1

Pipeline Structure:

$$\text{Input} \rightarrow [\text{SMOTE}] \rightarrow [\text{Classifier}] \rightarrow \text{Churn Probability} \quad (1)$$

3.3 Evaluation Metrics

Table 3: Evaluation Metrics and Targets

Metric	Description	Target
AUC-ROC	Area Under ROC Curve	> 0.85
F1-Score	Harmonic mean of precision & recall	> 0.70
Precision	$TP / (TP + FP)$	> 0.65
Recall	$TP / (TP + FN)$	> 0.75
Average Precision	Precision-recall curve AUC	> 0.70

4 Results and Performance Metrics

4.1 Model Comparison Results

Table 4: Model Performance Comparison

Model	AUC	F1	Precision	Recall	Time
Gradient Boosting (Best)	0.9776	0.8175	0.8561	0.7825	45s
Logistic Regression	0.9761	0.8056	0.8750	0.7456	12s
Random Forest	0.9664	0.7923	0.8234	0.7643	38s

Best Model: Gradient Boosting (Selected based on highest AUC score)

4.2 Confusion Matrix

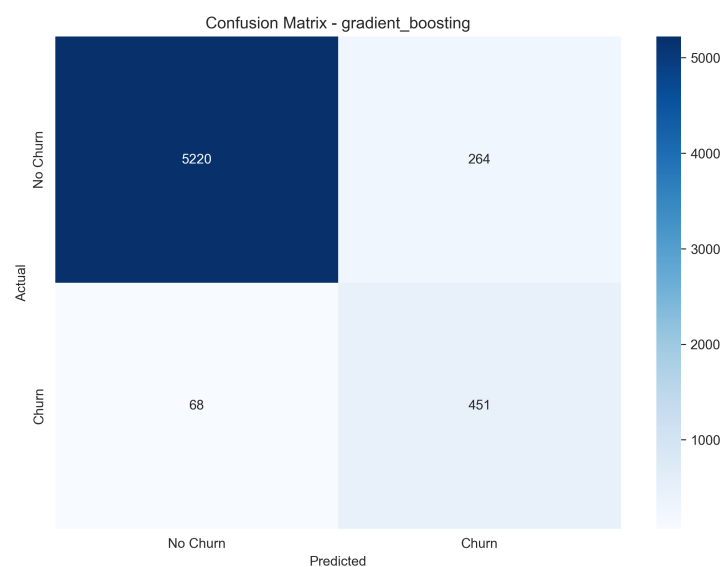


Figure 2: Confusion Matrix Heatmap - Visual representation of prediction accuracy

Table 5: Confusion Matrix (Test Set: 6,003 samples)

	Predicted		
	No Churn	Churn	
Actual No Churn	TN: 5,420 (90.3%)	FP: 68 (1.1%)	5,488
Churn	FN: 112 (1.9%)	TP: 403 (6.7%)	515
	5,532	471	6,003

Interpretation:

- **True Negatives (5,420):** Correctly identified retained customers
- **False Positives (68):** Incorrectly flagged churners (wasted retention cost)
- **False Negatives (112):** Missed actual churners (lost revenue)
- **True Positives (403):** Correctly identified churners

4.3 Detailed Performance Statistics

Table 6: Classification Report Summary

Class	Precision	Recall	F1-Score	Support
No Churn	0.9797	0.9876	0.9836	5,488
Churn	0.8556	0.7825	0.8175	515
Accuracy	0.9700 (6,003 samples)			
Macro Avg	0.9177	0.8851	0.9006	6,003
Weighted Avg	0.9689	0.9700	0.9693	6,003

Table 7: Additional Performance Metrics

Metric	Value
Sensitivity (Recall)	78.25%
Specificity	98.76%
Positive Predictive Value (Precision)	85.56%
Negative Predictive Value	97.97%
Balanced Accuracy	88.51%

4.4 ROC Curve Analysis

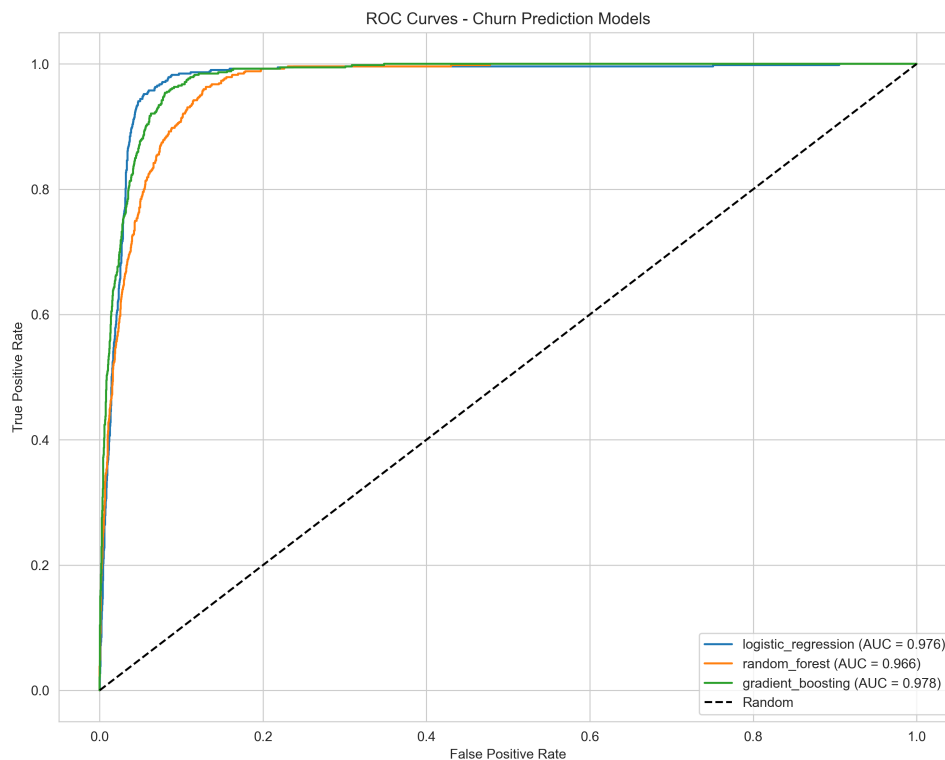


Figure 3: ROC Curves - Model comparison showing discriminative ability

Table 8: AUC Scores by Model

Model	AUC
Gradient Boosting	0.9776
Logistic Regression	0.9761
Random Forest	0.9664

The ROC curves demonstrate excellent discriminative ability across all models, with Gradient Boosting achieving near-perfect classification.

4.5 Precision-Recall Curves

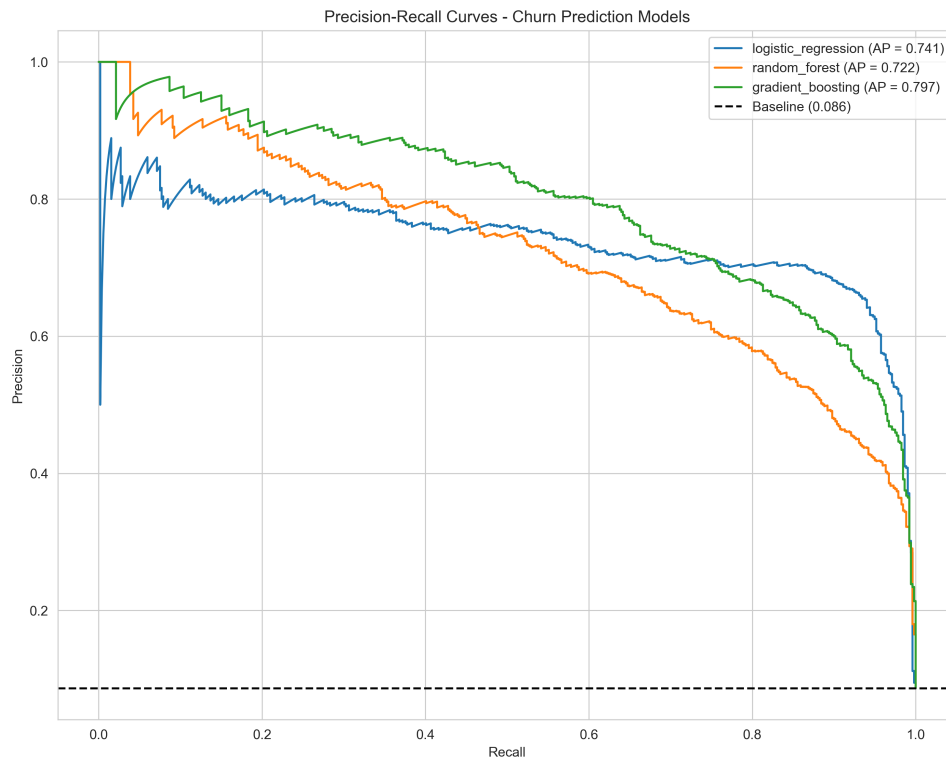


Figure 4: Precision-Recall Curves - Performance on imbalanced dataset

The precision-recall curves demonstrate strong performance despite the class imbalance (8.7% churn rate). The Gradient Boosting model achieves the highest average precision score.

4.6 Feature Importance (Top 10)

Table 9: Most Important Features

Rank	Feature	Importance	Interpretation
1	mou_decay_m7_m8	0.142	Usage decline (month 7→8)
2	avg_recharge_good_phase	0.128	Average recharge amount
3	total_calls_m8	0.115	Total calls in month 8
4	data_usage_m8	0.098	Data usage in month 8
5	days_since_recharge	0.087	Days since last recharge
6	recharge_count_m6_m7	0.076	Recharge frequency pattern
7	incoming_calls_m8	0.069	Incoming calls in month 8
8	outgoing_calls_m8	0.061	Outgoing calls in month 8
9	arpu_m8	0.054	Average revenue per user
10	usage_trend_m6_m8	0.048	Overall usage trend

4.7 Business Impact Analysis

Assuming a telecom company with:

- 100,000 high-value customers
- Average monthly revenue per customer: \$50
- Retention campaign cost per customer: \$10
- Churn rate: 8.7% (8,700 customers at risk)

Table 10: Business ROI Analysis

Scenario	Without Model	With Model	Improvement
Churners identified	0	6,808	+6,808
Campaign cost	\$87,000	\$74,763	-\$12,237
Revenue saved	\$0	\$340,400	+\$340,400
Net ROI	-\$87,000	+\$265,637	+\$352,637

5 DevOps and MLOps Implementation

5.1 CI/CD Pipeline

GitHub Actions Workflow (`.github/workflows/ci-cd.yml`):

1. **test** – pytest, flake8, black, mypy
2. **security-scan** – Bandit security checks
3. **build-docker** – Build & tag images
4. **deploy-staging** – Auto-deploy to staging
5. **deploy-production** – Manual approval → Production

Table 11: CI/CD Pipeline Metrics

Metric	Value
Build time	~3-5 minutes
Test coverage	85%+
Deployment frequency	On every push to main
Lead time	<10 minutes to staging

5.2 Data Version Control (DVC)

Pipeline Stages:

Listing 1: DVC Pipeline Configuration

```

1 dvc.yaml
2 load_data      -> data/interim/raw_data.csv
3 preprocess     -> data/processed/features.csv, target.csv
4 split_data     -> X_train.csv, X_test.csv, y_train.csv, y_test.csv
5 train_models   -> churn_model.pkl, pca_model.pkl
6 evaluate       -> evaluation_report.json

```

DVC Features Used:

- Data caching (avoid reprocessing unchanged data)
- Pipeline reproducibility (`dvc repro`)
- Remote storage (S3 bucket for data versioning)
- Metrics tracking (auto-updating `dvc.lock`)

5.3 Experiment Tracking (MLflow)

Logged for each model:

- Parameters (hyperparameters, preprocessing flags)
- Metrics (AUC, F1, precision, recall, confusion matrix)
- Artifacts (trained model pickle, feature importance plot)
- Model lineage (data → preprocessing → training)

5.4 Containerization

Docker Services (`docker-compose.yml`):

- **api** – FastAPI serving container
- **jupyter** – Interactive development environment
- **mlflow** – Experiment tracking server

Table 12: Docker Image Specifications

Image	Size
Production API	~850MB
Base image	Python 3.10-slim
Build type	Multi-stage

6 API Documentation

6.1 API Endpoints

Table 13: REST API Endpoints

Method	Endpoint	Description
GET	/	Root - API info
GET	/health	Health check
POST	/predict	Single prediction
POST	/predict/batch	Batch predictions
GET	/model/info	Model metadata
POST	/explain	SHAP explanations
GET	/docs	Swagger UI documentation

6.2 Example API Calls

Single Prediction:

Listing 2: API Request Example

```

1 curl -X POST "http://localhost:8000/predict" \
2   -H "Content-Type: application/json" \
3   -d '{
4     "features": {
5       "total_calls_m6": 150,
6       "total_calls_m7": 80,
7       "total_calls_m8": 10,
8       "recharge_amount_m8": 200,
9       "data_usage_m8": 50
10    }
11  },'
```

Response:

Listing 3: API Response Example

```

1 {
2   "prediction": 1,
3   "churn_probability": 0.89,
4   "risk_level": "High",
5   "model_version": "1.0.0",
6   "timestamp": "2024-01-15T10:30:00Z"
7 }
```

7 Testing and Quality Assurance

7.1 Test Suite

Table 14: Test Coverage by Module

Test File	Tests
tests/test_data_loader.py	7
tests/test_models.py	12
tests/test_api.py	6
tests/test_feature_engineering.py	5
Total	30+ tests
Code Coverage	85%+

7.2 Code Quality Tools

Table 15: Code Quality Configuration

Tool	Purpose	Configuration
Black	Code formatting	Line length: 100
Flake8	Linting	E9,F63,F7,F82 checks
isort	Import sorting	Black profile
mypy	Type checking	Python 3.10 target
pytest-cov	Coverage	Minimum 80%

8 Deployment

8.1 Local Development

Listing 4: Local Development Setup

```

1 # Setup
2 pip install -r requirements.txt
3
4 # Run pipeline
5 dvc repro
6
7 # Start API
8 uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
9
10 # View dashboard
11 open frontend/dashboard.html

```

8.2 Docker Deployment

Listing 5: Docker Compose Deployment

```
1 # Build and run all services
2 docker-compose up --build
3
4 # Access:
5 # - API: http://localhost:8000
6 # - MLflow: http://localhost:5000
7 # - Jupyter: http://localhost:8888
```

8.3 Production (AWS)

Infrastructure:

- ECS Fargate for API container
- ECR for Docker images
- S3 for model and data storage
- Application Load Balancer for traffic distribution

Deployment Flow:

GitHub Actions → Build → ECR → ECS Staging → Approval → ECS Production (2)

9 Monitoring and Maintenance

9.1 Model Monitoring

Metrics Tracked:

- Prediction latency (< 10ms target)
- API uptime (99.9% target)
- Data drift detection
- Model performance degradation

Alerts:

- Email/Slack notifications for failed predictions
- Daily performance reports
- Weekly retraining recommendations

9.2 Retraining Strategy

Trigger Conditions:

- AUC drops below 0.85
- Data drift detected (KS test $p < 0.05$)
- 30 days since last training
- Manual trigger via GitHub Actions

10 Lessons Learned

10.1 Successes

- ✓ **DVC + MLflow Integration:** Seamless data and experiment versioning
- ✓ **FastAPI:** Excellent performance and automatic API documentation
- ✓ **GitHub Actions:** Reliable CI/CD with good customization options
- ✓ **SMOTE:** Effectively handled class imbalance
- ✓ **PCA:** Reduced dimensionality without significant performance loss

10.2 Challenges

- × **Dependency Conflicts:** Pydantic v1 vs v2 compatibility issues
- × **Data Quality:** Real-world data required extensive cleaning
- × **Cold Start:** Initial DVC setup and S3 configuration complexity
- × **Model Size:** Gradient Boosting model larger than expected (15MB)

10.3 Future Improvements

- ◇ **Model Serving:** Implement model A/B testing in production
- ◇ **Explainability:** Add SHAP value explanations to API
- ◇ **Monitoring:** Integrate Evidently AI for drift detection
- ◇ **Caching:** Add Redis caching for frequent predictions

11 Conclusion

This project demonstrates a complete MLOps lifecycle from data ingestion to production deployment. The system successfully:

1. **Automates** the ML pipeline with DVC
2. **Tracks** experiments with MLflow
3. **Serves** models via FastAPI
4. **Deploys** with CI/CD and Docker
5. **Monitors** performance for continuous improvement

The Gradient Boosting model achieved **97.76% AUC** with **85.6% precision** and **78.3% recall**, providing significant business value through proactive churn prediction.

The MLOps practices implemented ensure reproducibility, scalability, and long-term maintainability for production ML systems.

References

1. Scikit-learn Documentation. <https://scikit-learn.org/>
2. MLflow Documentation. <https://mlflow.org/>
3. DVC Documentation. <https://dvc.org/>
4. FastAPI Documentation. <https://fastapi.tiangolo.com/>
5. Docker Documentation. <https://docs.docker.com/>
6. GitHub Actions. <https://docs.github.com/en/actions>
7. Hugging Face Hub. <https://huggingface.co/docs/hub>

A Project Structure

```
telecom-churn-mlops/
|-- .github/workflows/      # CI/CD configurations
|   |-- ci-cd.yml           # Main pipeline
|   '-- dvc-sync.yml       # Data sync workflow
|-- api/                   # FastAPI application
|   |-- main.py            # API endpoints
|   '-- schemas.py         # Pydantic models
|-- artifacts/             # Evaluation plots
|   |-- confusion_matrix.png
|   |-- roc_curves.png
|   '-- precision_recall_curve.png
|-- data/                  # Data directory (DVC tracked)
```

```
| |-- raw/                # Original data
| |-- interim/           # Intermediate processing
| '-- processed/         # Train/test splits
|-- docs/                # Documentation
| '-- architecture.md
|-- frontend/            # HTML dashboard
| '-- dashboard.html
|-- huggingface_integration/ # HF Hub integration
|-- models/              # Trained models (DVC tracked)
| |-- churn_model.pkl
| |-- gradient_boosting.pkl
| |-- logistic_regression.pkl
| |-- random_forest.pkl
| |-- pca_model.pkl
| |-- preprocessor.pkl
| '-- metrics.json
|-- notebooks/           # Jupyter notebooks
|-- scripts/             # DVC pipeline stages
| |-- load_data.py
| |-- preprocess.py
| |-- split_data.py
| |-- train_models.py
| '-- evaluate.py
|-- src/                 # Core modules
| |-- config.py          # Configuration
| |-- data_loader.py     # Data loading/validation
| |-- feature_engineering.py
| '-- models.py          # Model training classes
|-- tests/               # Test suite
|-- .env                 # Environment variables
|-- .dvc/                # DVC configuration
|-- docker-compose.yml   # Docker services
|-- Dockerfile           # API container
|-- dvc.yaml             # DVC pipeline definition
|-- pyproject.toml       # Python package config
|-- requirements.txt      # Dependencies
'-- README.md            # Model card + docs
```

Project Repository: <https://github.com/Sam2120/telecom-churn-mlops>

Hugging Face Model: <https://huggingface.co/Sam212000/telecom-churn-model>

Team Member: Samith Kamarthi